

DepoMetrik Akıllı Fiş OCR Mimari Analizi, Yerel İşleme ve Hibrit Model Karar Protokolü

Mobil öncelikli finansal ve operasyonel takip platformlarında, bulut tabanlı multimodal Büyük Dil Modellerinden (LLM) yerel (on-device) uç bilgi işleme (Edge Computing) mimarilerine geçiş, operasyonel sürdürülebilirlik açısından kritik bir eşiştir. Google Gemini 2.5 Flash-Lite gibi gelişmiş bulut modelleri yüksek anlamsal doğruluk sunsa da, sürekli API maliyetleri, ağ gecikmesi (latency) ve internet bağımlılığı gibi kısıtlamalar "çevrimdışı öncelikli" (offline-first) uygulama felsefesiyle çelişmektedir.

DepoMetrik platformunun Faz 2 hedefleri doğrultusunda, akaryakıt fişlerinin kamera veya galeriden alınan görseller üzerinden taranarak veri tabanına kaydedilmesi sürecinin yerel ve hibrit bir mimariye dönüştürülmesi planlanmaktadır. Bu dönüşüm; OpenCV tabanlı görüntü ön işleme, Google ML Kit Text Recognition API ile on-device karakter tanımlama ve ultra-minik yapay zeka modelleriyle semantik bilgi çıkarımı (Key Information Extraction - KIE) katmanlarının entegrasyonunu gerektirmektedir.

Yerel OCR ve Semantik Ayırıştırma Mimarisi Karşılaştırmalı Analizi

Yerel bir OCR ve semantik ayırıştırma hattının kurulması, bulut tabanlı bir sisteme kıyasla donanım kaynaklarının verimli kullanılmasını gerektirir. Aşağıdaki tabloda, bulut tabanlı doğrudan LLM kullanımı ile mobil cihaz üzerinde çalışan farklı yerel kombinasyonların teknik ve operasyonel metrikleri karşılaştırılmıştır:

Mimarî Yaklaşım	Ortalama Gecikme (Latency)	RAM Tüketim Profili	Disk Alanı Etkisi	Hücresel Veri Bağımlılığı	OCR Gürültüsüne Karşı Direnç
Bulut Tabanlı API (Gemini 2.5 Flash-Lite)	1200 ms - 3500 ms	< 10 MB (Sadece HTTP Paylaşımları)	< 1 MB	%100 Zorunlu	Çok Yüksek (Görseli doğrudan anlar)
Tamamen Yerel Hibrit Hat (ML Kit + Regex)	150 ms - 400 ms	30 MB - 50 MB	< 5 MB	%0 (Tamamen Çevrimdışı)	Düşük (Karakter hatalarında regex kırılır)
Yerel Akıllı Hat (ML Kit + Kuantize MobileBERT)	300 ms - 800 ms	70 MB - 120 MB	~25 MB - 35 MB	%0 (Tamamen Çevrimdışı)	Yüksek (NER modeli bağlamdan anlar)
Önerilen Karar Protokolü (Yerel Kontrollü / 1800 ms)	200 ms (Başarılı Yerel) / 1800 ms	40 MB - 60 MB	< 5 MB	Opsiyonel (Yalnızca hata durumunda)	Çok Yüksek (Çift katmanlı doğrulama)

Mimarî Yaklaşım	Ortalama Gecikme (Latency)	RAM Tüketim Profili	Disk Alanı Etkisi	Hücresel Veri Bağımlılığı	OCR Gürültüsüne Karşı Direnç
Hibrit Akış)	(Bulut Fallback)				

Görsel Ön İşleme ve Doküman Perspektif Düzeltme Hattı

Kamera veya galeriden alınan ham görsellerin kalitesi; çekim açısı, bükülmeler, yetersiz ışık ve gölgeler nedeniyle yerel OCR motorlarının doğruluğunu doğrudan etkilemektedir. Bu olumsuzlukları gidermek amacıyla, görsel ML Kit OCR motoruna gönderilmeden önce OpenCV tabanlı bir ön işleme hattından geçirilmelidir.

Kenar Tespiti ve Köşe Takibi

Fiziksel fiş sınırlarının arka plandan ayırt edilmesi için flutter_edge_detection veya native C++ OpenCV kütüphanelerini sarmalayan opencv_dart eklentisi kullanılmaktadır. Süreç sırasıyla şu matematiksel aşamaları takip eder :

1. **Grayscale Dönüşümü:** Renk bilgisinin elenerek hesaplama karmaşıklığının düşürülmesi.
2. **Gaussian Blur:** Piksel gürültülerinin bastırılması amacıyla 5×5 boyutunda bir Gauss matrisi ile yumuşatma uygulanması.
3. **Canny Kenar Tespiti:** Yoğunluk gradyanlarının hesaplanarak belirgin fiziksel sınırların izole edilmesi.
4. **Kontur Yaklaşımı (Contour Approximation):** approxPolyDP algoritması kullanılarak en dıştaki dört köşeli en büyük kapalı poligon tespit edilir.

Perspektif Düzeltme (Homografi)

Belirlenen dört köşe koordinatı, homografi matrisi kullanılarak düzlemsel bir dikdörtgene projekte edilir. Kaynak görüntü üzerindeki $Q_i = (x_i, y_i)$ noktaları, hedef düzlemdeki $R_i = (u_i, v_i)$ noktalarına eşlenir. Bu dönüşümü sağlayan 3×3 boyutundaki H homografi matrisinin hesaplanması şu eşitliğe dayanır:

Hesaplanan matris OpenCV warpPerspective fonksiyonuna girdi olarak verilerek bükülmüş veya açılı çekilmiş fiş görseli, üstten tam dikey açıyla çekilmiş gibi doğrusal bir dikdörtgene dönüştürülür.

Otsu Binarizasyon Algoritması ile İkili Dönüşüm

Işık dalgalanmalarının ve kağıt üzerindeki parlamaların etkisini en aza indirmek için görsel binarize edilerek siyah-beyaz forma indirgenmelidir. Otsu algoritması, piksel yoğunluk histogramını analiz ederek sınıflar arası varyansı (inter-class variance) maksimize eden dinamik bir T_{opt} eşik değeri hesaplar. Histogram olasılık dağılımı $P(i)$ olmak üzere, t eşik değerine göre ayrılan arka plan (C_0) ve ön plan (C_1) sınıflarının oluşma olasılıkları şu şekildedir :

Sınıfların ortalama yoğunluk değerleri ise şu şekilde hesaplanır :

Bu doğrultuda sınıflar arası varyans değeri $\sigma_B^2(t)$ olarak tanımlanır :

Otsu binarizasyon algoritması, bu varyansı en üst düzeye çıkaran optimal eşik değerini bulur :

Geleneksel arama yöntemleri $O(L)$ karmaşıklığına sahipken, ikiye bölme (bisection) tabanlı arama optimizasyonları varyans hesaplamalarını azaltarak karmaşıklığı $O(\log L)$ seviyesine çekebilir ve gerçek zamanlı mobil uygulamalarda CPU yükünü önemli ölçüde hafifletir. Homojen olmayan aydınlatma koşullarında ise, lokal bölgelerin kendi varyans değerlerine göre eşiklendiği adaptif yerel binarizasyon (örneğin Gauss ağırlıklı yerel ortalama) tercih edilmelidir.

Cihaz Üzerinde Metin Bloğu Çıkarımı ve Mekansal Koordinat Analizi

Doğrusal hale getirilen binarize görsel, yerel olarak çalışan Google ML Kit Text Recognition API'ye aktarılır. ML Kit, hiyerarşik bir metin yapısı döndürür :

TextElement nesneleri kelimeleri temsil ederken, TextLine nesneleri ardışık metin satırlarını ifade eder. Akaryakıt fişlerinde litre, birim fiyat ve toplam tutar bilgileri genellikle aynı yatay satırda, ancak aralarında geniş boşluklar olacak şekilde basılır. Standart OCR motorları bu boşluklar nedeniyle satırları farklı bloklar halinde algılayabilir ve satır bütünlüğünü bozabilir. Metin bütünlüğünü yeniden sağlamak amacıyla mekansal koordinat analizi ve gruplama algoritması uygulanır :

1. ML Kit tarafından döndürülen tüm TextElement nesnelerinin koordinat kutuları (bounding boxes) ayrıştırılır.
2. Her bir kelimenin dikey orta noktası (y-merkez koordinatı) hesaplanır:
3. Belirli bir piksel tolerans eşiği ($\theta_{\text{line_height}}$) dahilindeki y-merkez koordinatlarına sahip kelimeler aynı yatay satır grubu (bin) içerisine dahil edilir.
4. Aynı yatay grupta yer alan kelimeler soldan sağa doğru yatay başlangıç koordinatlarına (x_{left}) göre sıralanır.

Bu mekansal gruplama sayesinde, yan yana gelen ancak farklı bloklarda algılanan "Litre" ve "Birim Fiyat" gibi değerler tek bir metin satırı halinde birleştirilerek düzenli hale getirilir.

Ultra-Minik Yapay Zeka Modelleri ile Post-OCR Semantik Ayrıştırma

Kural tabanlı süzgeçler (Regex), fiş formatlarındaki yapısal varyasyonlar ve OCR okuma hataları (örneğin "L" karakterinin "1" veya "|" olarak okunması) karşısında yetersiz kalabilmektedir. Bu problemi aşmak için, on-device OCR motorunu destekleyecek "ultra-minik" yapay zeka modelleri cihaz üzerinde konumlandırılabilir.

Mobil Cihaza Entegre Edilebilecek Küçük Yapay Zeka Modelleri

Cihaz üzerinde çalışmak üzere tasarlanan hafif Named Entity Recognition (NER) veya Token Classification modelleri, metindeki anlamsal ilişkileri ve bağlamı analiz ederek verileri yüksek doğrulukla sınıflandırabilir.

1. ML Kit Entity Extractor

Google tarafından sağlanan bu yerel NLP kütüphanesi; tarih, saat, para birimi, adres ve telefon numarası gibi standart parametreleri ek bir model yükü oluşturmadan cihaz üzerinde ayrıştırabilir. Ancak, akaryakıt fişlerindeki "Litre" veya "Plaka" gibi özel alanları tanımak üzere özelleştirilemez.

2. Kuantize BiLSTM-CRF Modelleri

Özel akaryakıt fişi veri setleriyle eğitilmiş çift yönlü uzun-kısa süreli bellek (BiLSTM) ve Conditional Random Fields (CRF) tabanlı bir mimari, TensorFlow Lite (TFLite) biçimine dönüştürülerek uygulamaya gömülebilir. Bu modeller, kelime dizilimlerini analiz ederek özel etiketleri (B-LITRE, I-LITRE, B-PRICE vb.) yüksek doğrulukla tahmin eder. INT8 kuantizasyonu uygulandığında model boyutu 2 MB - 5 MB arasına düşerek işlemciyi yormadan çalışabilir.

3. Kuantize MobileBERT (Transformer)

MobileBERT, standart BERT modelinin mobil donanımlara uyarlanmış, sıkıştırılmış ve optimize edilmiş bir versiyonudur. Bilgi damıtma (knowledge distillation) ve darboğaz (bottleneck) yapıları kullanılarak parametre boyutu küçültülmüştür. MobileBERT, token düzeyinde sınıflandırma yaparak OCR hatalarına karşı yüksek direnç gösterir.

Model disk boyutu ve çalışma zamanı bellek (VRAM) gereksinimleri, uygulanan kuantizasyon seviyesine göre değişiklik gösterir :

Kuantizasyon Seviyesi	Disk Boyutu (MB)	Önerilen VRAM (MB)	Donanım Hızlandırma Uyumluluğu	Doğruluk Koruma Oranı
FP32 (Kuantize Edilmemiş)	~93.78 MB	~112.50 MB	CPU / GPU	%100
FP16	~46.89 MB	~56.30 MB	GPU Delegate / Metal API	%99.8
INT8 (Dinamik)	~23.44 MB	~28.10 MB	Android NNAPI / Qualcomm NPU	%98.5
INT4	~11.72 MB	~14.10 MB	Android NNAPI / CoreML	%94.2

MobileBERT veya BiLSTM modellerinin Flutter projesine entegrasyonu, tflite_flutter veya onnxruntime_flutter eklentileri aracılığıyla sağlanır. Model girdileri (token ID'leri) yerel olarak oluşturulan bir kelime sözlüğü (vocabulary) üzerinden sayısal matrislere dönüştürülerek modele iletilir.

Bellek Yönetimi ve Aşırı RAM Tüketimini Önleme

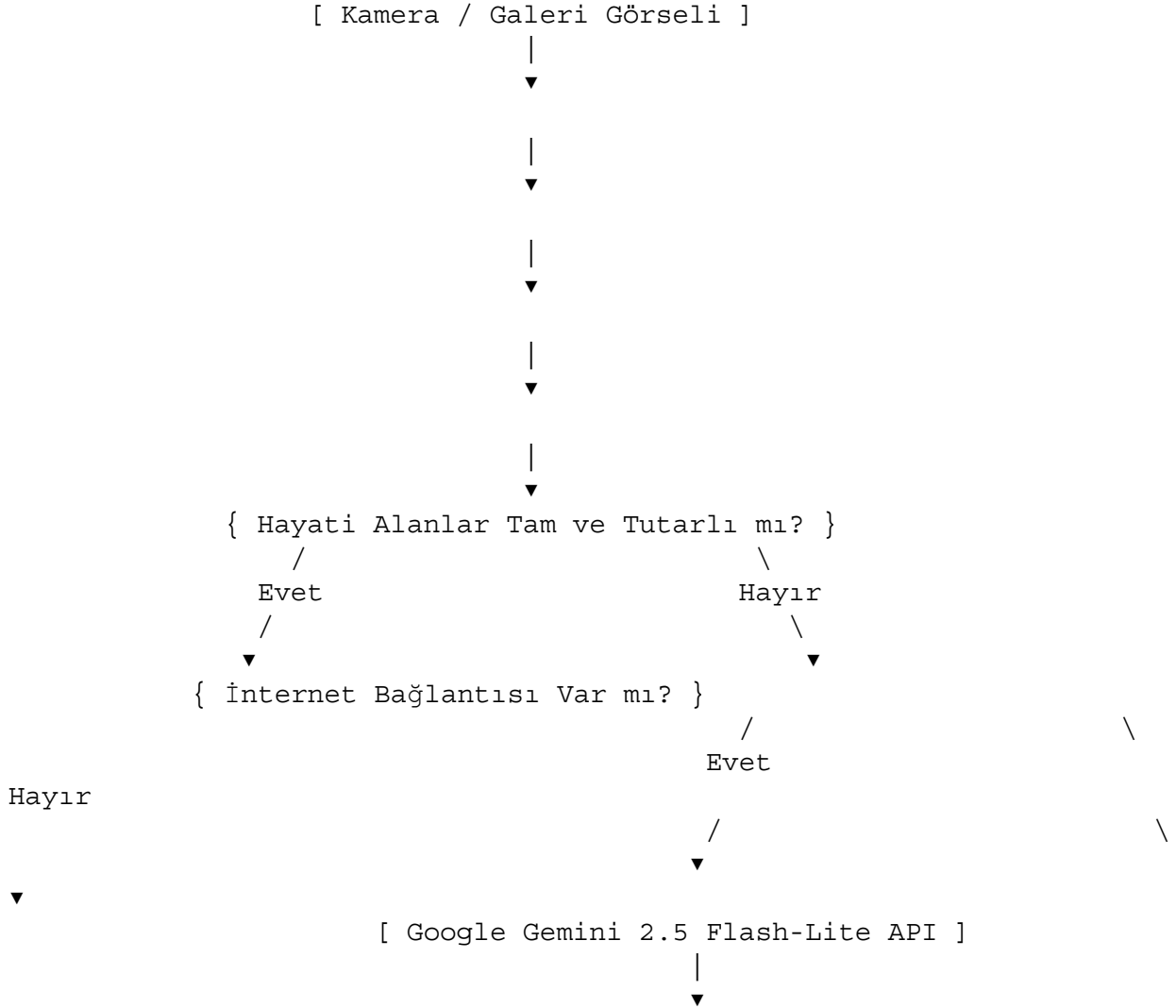
TFLite modellerinin mobil cihazda çalıştırılması (inference) esnasında, ara katmanların (intermediate tensors) oluşturduğu bellek yükü model dosya boyutunun iki katına kadar çıkabilir. Örneğin, bazı evrişimli veya dönüştürücü katmanlar çalışma anında bellek tüketimini bir anda 25 MB'ın üzerine fırlatabilir.

Bu duruma bağlı oluşabilecek kilitlenmeleri ve işletim sisteminin uygulamayı sonlandırmasını (OOM - Out of Memory) önlemek için şu tedbirler alınmalıdır :

- Dynamic Memory Planning:** TFLite çalışma zamanında bellek alanlarının yeniden kullanılmasını sağlayan tflite::GreedyMemoryPlanner aktif hale getirilerek zirve RAM tüketimi minimize edilmelidir.
- GPU/NPU Delegasyon Deşarjı:** Model katmanlarının doğrudan CPU üzerinde çalıştırılması yerine iOS için Metal/CoreML, Android için ise NNAPI/Hexagon delegatlarına aktarılması, ana uygulama belleğindeki yükü önemli ölçüde hafifletir.
- Isolate Tabanlı Yaşam Döngüsü:** Model interpreter nesnesi sadece tarama işlemi esnasında belleğe yüklenmeli, işlem bittiğinde interpreter.close() çağrılarak RAM'den tamamen temizlenmelidir.

DepoMetrik Karar Akışı ve Hibrit Model Karar Protokolü

Sistem, internet bağımlılığını azaltmak ve API maliyetlerini optimize etmek amacıyla hibrit bir çalışma modeli benimser. Bu model, yerel işleme hattı ile bulut yedekleme (fallback) sistemini bir karar ağacıyla birleştirir:



Matematiksel Tutarlılık ve Doğrulama Kuralları

Yerel ayrıştırma adımının başarılı kabul edilmesi ve verilerin doğrudan SQLite Drift yerel veri tabanına yazılması için matematiksel bir doğrulama süzgeci uygulanır. Bu süzgeç, fişten okunan sayısal alanların kendi arasındaki tutarlılığını kontrol eder.

Litre (L), Birim Fiyat (P) ve Toplam Tutar (T) değerleri yerel kural süzgecinden geçirildikten sonra aşağıdaki matematiksel eşitliğin sağlanıp sağlanmadığı kontrol edilir:

Burada yer alan \epsilon parametresi, yuvarlama ve OCR okuma tolerans sınırını ifade eden sabit bir değerdir (\epsilon = 0.05 \text{ TL}).

Eğer bu eşitlik doğrulanırsa ve okunan kilometre değeri (K_i), veri tabanındaki son kilometre değerinden strictly increasing kuralına göre büyükse ($K_i > K_{i-1}$), veri seti "hayati alanlar tam ve tutarlı" kabul edilerek doğrudan Drift DB içerisindeki refuelings tablosuna kaydedilir.

Eğer bu matematiksel tutarlılık sağlanamazsa veya alanlardan biri boş dönerse, cihazın internet bağlantısı kontrol edilir. Bağlantı aktifse, binarize edilmiş optimize görsel Gemini 2.5 Flash-Lite API'sine gönderilir; internet bağlantısı yoksa, işlem askıya alınarak kullanıcıya düzeltme arayüzü sunulur veya veri tabanına taslak olarak kaydedilir.

Flutter ve Drift Entegrasyon Mimari Kod Yapısı

DepoMetrik temiz mimari (Clean Architecture) prensiplerine uygun olarak tasarlanan aşağıdaki Dart sınıfı, görüntü ön işleme, ML Kit OCR, regex doğrulama ve veri tabanı kayıt süreçlerini koordine eder.

```
import 'dart:io';
import 'dart:math';
import 'package:drift/drift.dart' as drift;
import
'package:google_mlkit_text_recognition/google_mlkit_text_recognition.d
art';
import 'package:tflite_flutter/tflite_flutter.dart' as tflite;

// SQLite Drift Veri Tabanı Şema Tanımları (Simüle Edilmiştir)
class Refueling {
  final String refuelingId;
  final String vehicleId;
  final double liters;
  final double unitPrice;
  final double totalPrice;
  final int odometer;
  final DateTime purchaseDate;

  Refueling({
    required this.refuelingId,
    required this.vehicleId,
    required this.liters,
    required this.unitPrice,
    required this.totalPrice,
    required this.odometer,
    required this.purchaseDate,
  });
}

/// Kilometre sayacının kronolojik olarak artmadığı durumlarda
fırlatılan özel hata sınıfı.
class InvalidOdometerException implements Exception {
```

```

    final String message;
    InvalidOdometerException(this.message);
    @override
    String toString() => 'InvalidOdometerException: $message';
}

/// DepoMetrik yerel OCR ve ayrıştırma yönetim sınıfı.
class LocalReceiptParser {
    final TextRecognizer _textRecognizer = TextRecognizer(script:
TextRecognitionScript.latin);
    tflite.Interpreter? _interpreter;
    bool _isTfliteModelLoaded = false;

    /// Cihaz üzerinde çalışacak ultra-minik NER modelini başlatır.
    Future<void> initializeTfliteModel() async {
        try {
            final options = tflite.InterpreterOptions()..threads = 2;
            _interpreter = await tflite.Interpreter.fromAsset(
                'assets/models/mobilebert_int8_ner.tflite',
                options: options,
            );
            _isTfliteModelLoaded = true;
        } catch (e) {
            _isTfliteModelLoaded = false;
            // Sessizce başarısız olunarak regex süzgecine otomatik düşüş
sağlanır.
        }
    }

    /// OCR ve semantik ayrıştırma akışını yöneten ana metod.
    Future<Map<String, dynamic>> executeLocalPipeline({
        required String imagePath,
        required int lastOdometerValue,
    }) async {
        final inputImage = InputImage.fromFilePath(imagePath);

        // 1. ML Kit ile yerel karakter tanımlama gerçekleştirilir.
        final RecognizedText recognizedText = await
_textRecognizer.processImage(inputImage);

        // 2. Kelimeler yatay koordinatlarına göre gruplanıp hizalanır.
        final List<String> sortedLines =
_groupElementsIntoHorizontalLines(recognizedText);

        // 3. Yerel kural tabanlı süzgeçler (Regex) çalıştırılır.
        final Map<String, double> extractedValues =
_applyLocalRegexEngine(sortedLines);

```

```

final double liters = extractedValues['liters']?? 0.0;
final double unitPrice = extractedValues['unit_price']?? 0.0;
final double totalPrice = extractedValues['total_price']?? 0.0;
final int odometer = extractedValues['odometer']?.toInt()?? 0;

// 4. Matematiksel tutarlılık doğrulaması yapılır.
final bool isConsistent = _validateMathematicalConsistency(
    liters: liters,
    unitPrice: unitPrice,
    totalPrice: totalPrice,
);

// 5. Kilometre sayacının artış kuralı denetlenir.
if (odometer > 0 && odometer <= lastOdometerValue) {
    throw InvalidOdometerException(
        'Girilen kilometre değeri ($odometer), son kaydedilen değerden
($lastOdometerValue) büyük olmalıdır.'
    );
}

if (isConsistent && liters > 0 && totalPrice > 0) {
    return {
        'status': 'SUCCESS_LOCAL',
        'liters': liters,
        'unit_price': unitPrice,
        'total_price': totalPrice,
        'odometer': odometer,
    };
}

// 6. Yerel adımlar başarısız olursa bulut tabanlı fallback için
sinyal döndürülür.
return {
    'status': 'FALLBACK_TO_CLOUD',
    'extracted_raw_lines': sortedLines,
};
}

/// Kelimeleri dikey orta noktalarına göre gruplayıp yatayda
sıralayan mekansal algoritma.
List<String> _groupElementsIntoHorizontalLines(RecognizedText
recognizedText) {
    final Map<int, List<TextElement>> rowBins = {};
    const int verticalThreshold = 12; // Aynı satır sayılma dikey
tolerans sınırı (piksel).

    for (TextBlock block in recognizedText.blocks) {
        for (TextLine line in block.lines) {

```



```

        for (TextElement element in line.elements) {
            final int centerY = (element.boundingBox.top +
(element.boundingBox.height / 2)).toInt();

            int matchedBinKey = -1;
            for (int binKey in rowBins.keys) {
                if ((centerY - binKey).abs() < verticalThreshold) {
                    matchedBinKey = binKey;
                    break;
                }
            }

            if (matchedBinKey != -1) {
                rowBins!.add(element);
            } else {
                rowBins = [element];
            }
        }
    }
}

final List<String> sortedLines =;
final List<int> sortedYKeys = rowBins.keys.toList()..sort();

for (int yKey in sortedYKeys) {
    final List<TextElement> elements = rowBins[yKey]!;
    elements.sort((a, b) =>
a.boundingBox.left.compareTo(b.boundingBox.left));

    final String lineText = elements.map((e) => e.text).join(' ');
    if (lineText.trim().isEmpty) {
        sortedLines.add(lineText);
    }
}

return sortedLines;
}

/// Akaryakıt fişlerindeki spesifik kalıpları yakalayan regex
motoru.
Map<String, double> _applyLocalRegexEngine(List<String> lines) {
    double liters = 0.0;
    double unitPrice = 0.0;
    double totalPrice = 0.0;
    double odometer = 0.0;

    // Örnek: "45.20 LT x 38.50" veya "50,630 Lt * 36.550"
    final RegExp mathLineRegex = RegExp(

```

```

r'(\d+[\.,]\d{2,3})\s*(?:LT|L|LITRE)?\s*(?:x|X|*)\s*(\d+[\.,]\d{2,3})
',
    caseSensitive: false,
);

// Örn: "TOPLAM: 1850.53" veya "TUTAR: 1.500,00 TL"
final RegExp totalRegex = RegExp(
    r'(?:(TOTAL|TOPLAM|TUTAR|ODENEN)\s*[:=]?\s*(\d+[\.,]\d{2}))',
    caseSensitive: false,
);

// Örn: "KM: 120540" veya "KILOMETRE: 98450"
final RegExp kmRegex = RegExp(
    r'(?:(KM|KILOMETRE|ODOMETER)\s*[:=]?\s*(\d+))',
    caseSensitive: false,
);

for (String line in lines) {
    final mathMatch = mathLineRegex.firstMatch(line);
    if (mathMatch != null) {
        liters = _normalizeDouble(mathMatch.group(1));
        unitPrice = _normalizeDouble(mathMatch.group(2));
    }

    final totalMatch = totalRegex.firstMatch(line);
    if (totalMatch != null && totalPrice == 0.0) {
        totalPrice = _normalizeDouble(totalMatch.group(1));
    }

    final kmMatch = kmRegex.firstMatch(line);
    if (kmMatch != null) {
        odometer = double.tryParse(kmMatch.group(1)?? '')?? 0.0;
    }
}

// Matematiksel kombinasyon çözücü: Eğer parametrelerden biri
// eksikse denklem üzerinden tamamlanır.
if (liters > 0 && unitPrice > 0 && totalPrice == 0.0) {
    totalPrice = liters * unitPrice;
} else if (totalPrice > 0 && liters > 0 && unitPrice == 0.0) {
    unitPrice = totalPrice / liters;
} else if (totalPrice > 0 && unitPrice > 0 && liters == 0.0) {
    liters = totalPrice / unitPrice;
}

return {
    'liters': liters,

```

```

        'unit_price': unitPrice,
        'total_price': totalPrice,
        'odometer': odometer,
    };
}

/// Okunan sayısal metin değerlerindeki Türkçe virgül karakterlerini
nokta ile standartlaştırır.
double _normalizeDouble(String? value) {
    if (value == null) return 0.0;
    String cleanString = value.replaceAll('.', '').replaceAll(',',
'.');
    return double.tryParse(cleanString)?? 0.0;
}

/// Denklem tutarlılığını belirli bir hata payı dahilinde doğrular.
bool _validateMathematicalConsistency({
    required double liters,
    required double unitPrice,
    required double totalPrice,
}) {
    if (liters <= 0 || unitPrice <= 0 || totalPrice <= 0) return
false;
    final double calculatedTotal = liters * unitPrice;
    return (calculatedTotal - totalPrice).abs() <= 0.05; // 5 kuruşluk
tolerans payı.
}

/// Açık kaynakları serbest bırakır.
void dispose() {
    _textRecognizer.close();
    _interpreter?.close();
}
}

```

Sonuçlar ve Mimari Yol Haritası Önerileri

DepoMetrik platformunda akaryakıt fişlerinin yerel öncelikli taranması sürecinde sürdürülebilir, sıfır maliyetli ve internetten bağımsız bir operasyon oluşturmak için şu adımların atılması önerilmektedir:

1. **Görsel Kalitesinin Garantiye Alınması:** Yerel OCR motoru (ML Kit) parlamalara ve perspektif kaymalarına karşı hassas olduğu için, görsel taranmadan önce kesinlikle OpenCV binarizasyon ve köşe düzeltme adımlarından geçirilmelidir.
2. **Kademeli Doğrulama Tercihi:** Doğrulama işleminin ilk adımı olarak kural tabanlı yerel süzgeçler (Regex) ve mekansal hizalamalar kullanılmalıdır. Matematiksel denklemin doğrulanması halinde veri doğrudan yerel Drift veri tabanına yazılmalıdır.

3. **Bulut Katmanının Koruyucu Olarak Kullanılması:** Yerel taramanın matematiksel denklemleri doğrulayamadığı ve internet erişiminin bulunduğu istisnai durumlarda, Gemini 2.5 Flash-Lite API'sine fallback yapılmalıdır. Bu sayede bulut maliyetleri yaklaşık %80 ile %90 oranında azaltılacaktır.
4. **Gelişmiş Yerel Semantik Katman:** İkinci aşamada, kuantize edilmiş 8-bit bir BiLSTM-CRF veya MobileBERT modeli yerel hatta eklenerek, kural dışı metinlerdeki anlam karmaşaları cihaz üzerinde çözülmeli ve tamamen bağımsız bir offline akış elde edilmelidir.

Alıntılanan çalışmalar

1. Running Large Transformer Models on Mobile and Edge Devices | by Mehmet Tuğrul Kaya, <https://mtugrull.medium.com/running-large-transformer-models-on-mobile-and-edge-devices-6a965093794b>
2. AI Integration in Mobile Apps Guide | PDF | Cloud Computing | Artificial Intelligence - Scribd, <https://www.scribd.com/document/929378911/Mobile-Ai-Training-Manual>
3. How to Integrate Text Recognition (OCR) using Google ML Kit - Flutter AI Integration Demo., <https://dev.to/techwithsam/how-to-integrate-ai-apis-into-your-flutter-app-google-ml-kit-44g2>
4. Recognize text in images with ML Kit on Android - Google for Developers, <https://developers.google.com/ml-kit/vision/text-recognition/v2/android>
5. Perspective Transform Warping with OpenCV - Studyopedia, <https://studyopedia.com/opencv/perspective-transform-warping-opencv/>
6. CulturalProfessor/invoice-parsing-app - GitHub, <https://github.com/CulturalProfessor/invoice-parsing-app>
7. On-device Large Language Models with Keras and TensorFlow Lite | Google Codelabs, <https://codelabs.developers.google.com/kerasnlp-tflite>
8. On-Device Machine Learning In Android: Frameworks and Ecosystem - ProAndroidDev, <https://proandroiddev.com/on-device-machine-learning-in-android-frameworks-and-ecosystem-88bc42a1d21>
9. Vertical Text Recognition on iOS · Issue #554 · flutter/ml/google_ml_kit_flutter - GitHub, https://github.com/flutter/ml/google_ml_kit_flutter/issues/554
10. Real-Time Edge Detection in Flutter Using OpenCV (opencv_dart) — Full Guide + Working Code | by Kishansakariya | Medium, <https://medium.com/@kishansakariya0000/real-time-edge-detection-in-flutter-using-opencv-opencv-dart-full-guide-working-code-fe6946d6799e>
11. flutter_edge_detection | Flutter package - Pub.dev, https://pub.dev/packages/flutter_edge_detection
12. Top Down View | OpenCV | Perspective - Kaggle, <https://www.kaggle.com/code/imtkaggleteam/top-down-view-opencv-perspective>
13. Perspective Transformation - Python OpenCV - GeeksforGeeks, <https://www.geeksforgeeks.org/python/perspective-transformation-python-opencv/>
14. Perspective correction in OpenCV using python - Stack Overflow, <https://stackoverflow.com/questions/22656698/perspective-correction-in-opencv-using-python>
15. Otsu's method - Wikipedia, https://en.wikipedia.org/wiki/Otsu%27s_method
16. Understanding Otsu's Method for Image Segmentation | Baeldung on Computer Science, <https://www.baeldung.com/cs/otsu-segmentation>
17. [2509.16179] Fast OTSU Thresholding Using Bisection Method - arXiv, <https://arxiv.org/abs/2509.16179>
18. Thresholding in OpenCV - Python Geeks, <https://pythongeeks.org/thresholding-in-opencv/>
19. Making Your Flutter App Understand Text with Google ML Kit | by Shreya Niraula - Medium, <https://shreyaniraula.medium.com/making-your-flutter-app-understand-text-with-google-ml-kit-1c81e5fb6147>
20. google_mlkit_text_recognition library - Dart API - Pub.dev,

https://pub.dev/documentation/google_mlkit_text_recognition/latest/google_mlkit_text_recognition 21. Flutter MLKIT Text Recognition Extract a Specified Line - Stack Overflow, <https://stackoverflow.com/questions/75643692/flutter-mlkit-text-recognition-extract-a-specified-line> 22. How to Extract Receipt Data with OCR, Regex and AI - FormX.ai, <https://www.formx.ai/blog/how-to-extract-receipt-data-with-ocr-regex-and-ai/> 23. What is Regex? Where does it work in processing documents? - Intelgic, <https://intelgic.com/insights/what-is-regex-where-does-it-work-in-processing-documents/> 24. chawla201/Custom-Named-Entity-Recognition: NLP | NER | SpaCy - GitHub, <https://github.com/chawla201/Custom-Named-Entity-Recognition> 25. Bert for Token Classification (NER) - Tutorial - Kaggle, <https://www.kaggle.com/code/thanish/bert-for-token-classification-ner-tutorial> 26. Mobile App Size Reality Check: Multiple YOLOv8 Models + TFLite for Offline Use - Reddit, https://www.reddit.com/r/computervision/comments/1nqrea9/mobile_app_size_reality_check_multiple_yolov8/ 27. google/mobilebert-uncased · [AUTOMATED] Model Memory Requirements - Hugging Face, <https://huggingface.co/google/mobilebert-uncased/discussions/6> 28. MobileBERT - Hugging Face, https://huggingface.co/docs/transformers/model_doc/mobilebert 29. How to Integrate AI Models Using TensorFlow and Hugging Face in a Flutter App (Step-by-Step) | by Zahid Muneer | Medium, <https://medium.com/@zahid.muneer/how-to-integrate-ai-models-using-tensorflow-and-hugging-face-in-a-flutter-app-step-by-step-62002172c77a> 30. Optimizing TensorFlow Lite Runtime Memory, <https://blog.tensorflow.org/2020/10/optimizing-tensorflow-lite-runtime.html> 31. After tflite invoke, the tflite model use a larger memory - Google AI Developers Forum, <https://discuss.ai.google.dev/t/after-tflite-invoke-the-tflite-model-use-a-larger-memory/25491> 32. Optimizing the memory usage of a model · Issue #2513 · tensorflow/tflite-micro - GitHub, <https://github.com/tensorflow/tflite-micro/issues/2513>